

本文档内容可能会因产品更新而有所变动，请访问
<https://zitd5je6f7j.feishu.cn/wiki/GQLJwGNNfiCxlDkrwZLclmD4nWc> 获取最新版本。

适用软件版本	v2.0.5
文档更新时间	2025-09-26

1. 关于天轶 2.0 Pro 版

Pro 版天轶 2.0 整机分为上半身、下半身和一个底盘，具备多个自由度。头部拥有 3 个自由度，单手臂拥有 7 个自由度，包括肩关节、肘关节和腕关节。腰部有 2 个自由度。腿部拥有 2 个自由度，包括髋关节和膝关节。底盘有 2 个自由度，可 360 度自由旋转。整机共有 35 个自由度，使得机器人能够实现精确的运动和姿态控制。

1.1. 总体规格参数

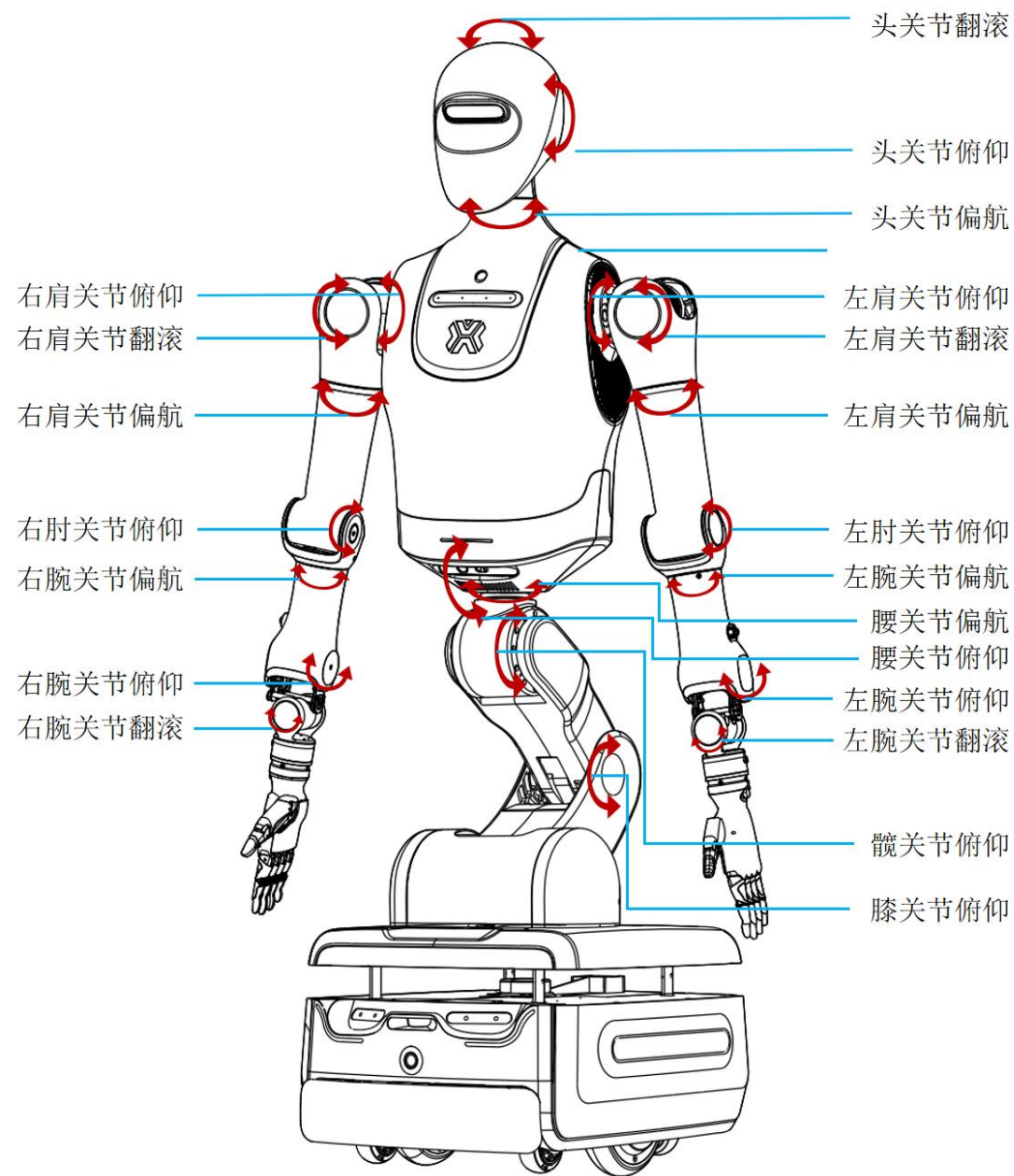
参数类型	参数名	说明
规格参数	产品高度	1300~1650mm
	手臂臂展	600mm
	产品净重	88.5kg
	产品颜色	白色+黑色
	产品材料	铝合金+工程塑料
	自由度	35 个一体化关节 (头 3 个, 臂 7 个*2, 手 6 个*2, 腰 2 个, 腿 2 个, 底盘 2 个)
电池参数	电池	电池类型：三元锂; 容量：15Ah 电压：48V
传感器参数	高精度六维力	有
	摄像头	头部：RGBD*1; 底盘：RGBD*2, RGB*1, 碰撞检测, 磁感应

本文档内容可能会因产品更新而有所变动，请访问

<https://zitd5je6f7j.feishu.cn/wiki/GQLJwGNNfiCxlDkrwZLclmD4nWc> 获取最新版本。

	语音模块	MIC 阵列：线性 MIC*4; 扬声器*1; 声卡*1; 720P RGB 单目相机*1
控制计算参数	运控计算单元	型号：Intel x86 i7-1355U; CPU 10 核 12 线程; 主频最高达 5GHz; 内存：16GB; SSD：256GB
	开发计算单元	型号：Orin AGX 64*1 算力：275 TOPS
	系统平台/软件	操作系统：Ubuntu22.04.4 LTS; 中间件：ROS
执行机构参数	机械臂	7 自由度手臂*2, 单手末端负载 4kg, 末端重复定位精度 $\pm 0.1\text{mm}$
	灵巧手	单手负载 3kg, 末端重复定位精度 $\pm 0.1\text{mm}$
通信参数	外部通讯能力	WiFi6, Ethernet, Bluetooth5.3;
	内部通信网络	全身 CAN/EtherCAT 总线通讯 控制速度：>400Hz

1.2. 全身关节参数

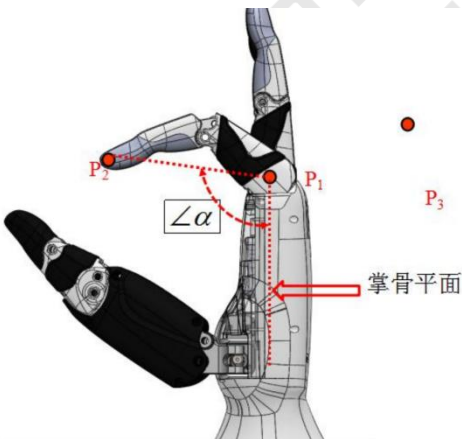
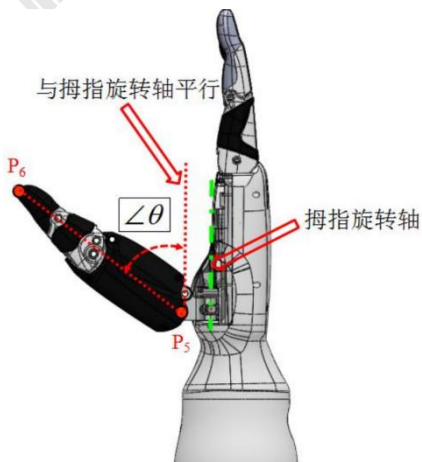


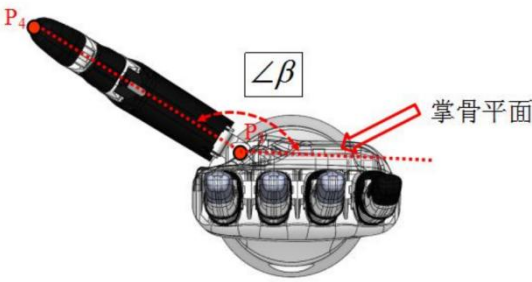
所属部位	名称	运动范围 (°)	扭矩系数 (Nm/A)	最大转速 (rpm)	KP/KD 范围	额定力矩 (Nm)	峰值力矩 (Nm)	关节名称
头部	头关节翻滚 Head Roll	-26°~+26°	2.16	64	Kp:0~2000 kd: 0~300	3	9	1
	头关节俯仰 Head Pitch	-25°~+25°	2.16	64	Kp:0~2000 kd: 0~300	3	9	2

	头关节偏航 Head Yaw	-90°~+90°	2.16	64	Kp:0~2000 kd: 0~300	3	9	3
腰部	腰关节偏航 Waist Yaw	-160° ~+180°	/	30	/	31	143	31
	腰关节俯仰 Waist Pitch	-45°~+120°	/	37.5	/	82	332	32
左臂	左肩关节俯仰 Left Shoulder Pitch	-170° ~+170°	3.207	88	Kp:0~2000 kd: 0~300	35	91	11
	左肩关节翻滚 Left Shoulder Roll	-15°~+150°	3.37	120	Kp:0~2000 kd: 0~300	20	60	12
	左肩关节偏航 Left Shoulder Yaw	-170° ~+170°	2.43	73	Kp:0~2000 kd: 0~300	10	30	13
	左肘关节俯仰 Left Elbow Pitch	-150°~+15°	2.43	73	Kp:0~2000 kd: 0~300	10	30	14
	左腕关节偏航 Left Wrist Yaw	-170° ~+170°	1.71	146	Kp:0~2000 kd: 0~300	5	15	15
	左腕关节俯仰 Left Wrist	-45°~+60°	2.16	72	Kp:0~2000 kd: 0~300	3	9	16

	Pitch							
	左腕关节翻滚 Left Wrist Roll	-95°~+75°	2.16	72	Kp:0~2000 kd: 0~300	3	9	17
右臂	右肩关节俯仰 Right Shoulder Pitch	-170° ~+170°	3.207	88	Kp:0~2000 kd: 0~300	35	91	21
	右肩关节翻滚 Right Shoulder Roll	-150°~+15°	3.37	120	Kp:0~2000 kd: 0~300	20	60	22
	右肩关节偏航 Right Shoulder Yaw	-170° ~+170°	2.43	73	Kp:0~2000 kd: 0~300	10	30	23
	右肘关节俯仰 Right Elbow Pitch	-150°~+15°	2.43	73	Kp:0~2000 kd: 0~300	10	30	24
	右腕关节偏航 Right Wrist Yaw	-170° ~+170°	1.71	146	Kp:0~2000 kd: 0~300	5	15	25
	右腕关节俯仰 Right Wrist Pitch	-45°~+60°	2.16	72	Kp:0~2000 kd: 0~300	3	9	26
	右腕关节翻滚	-75°~+95°	2.16	72	Kp:0~2000	3	9	27

	Right Wrist Roll				kd: 0~300			
腿部	髋关节俯仰 Hip Pitch	+13°~+80°	/	37.5	/	332	332	51
	膝关节俯仰 Knee Pitch	+26°~+160°	/	37.5	/	332	332	52

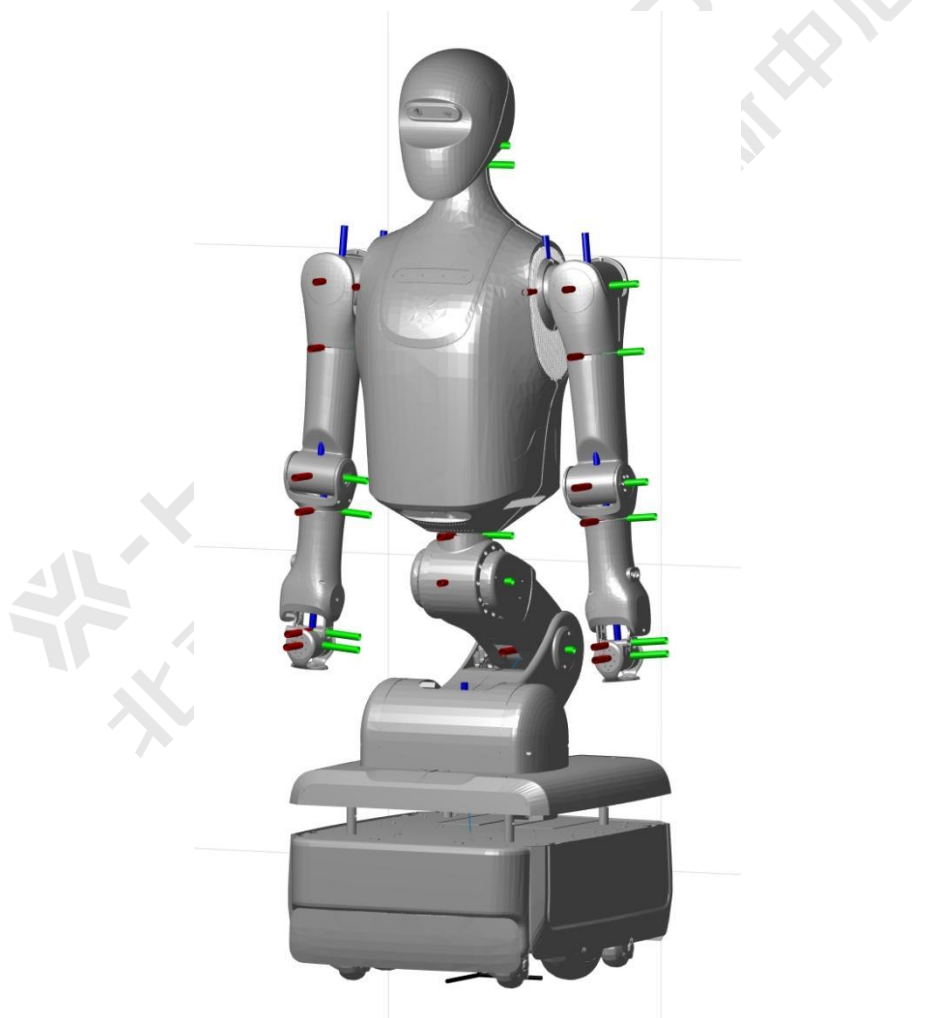
角度	运动范围 (°)	图例说明
小拇指 无名指 中指 食指	+19°~+176.7°	 掌骨平面
大拇指弯曲角度	-13°~+53.6°	 与拇指旋转轴平行 拇指旋转轴

大拇指旋转角度	+90°~+165°	
---------	------------	--

1.3. 整机 urdf, 坐标系, 关节旋转轴与关节零点

1.3.1. 关节零点

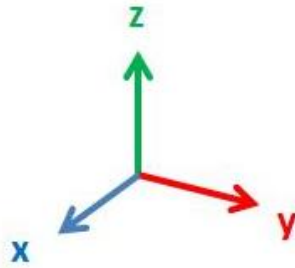
如下图所示：



1.3.2. 坐标系

1. 整机坐标系-X 轴 (Roll) 以机器人前方为正方向；

2. 整机坐标系-Y 轴 (Pitch) 以机器人左方为正方向;
3. 整机坐标系-Z 轴 (Yaw) 以机器人上方为正方向;

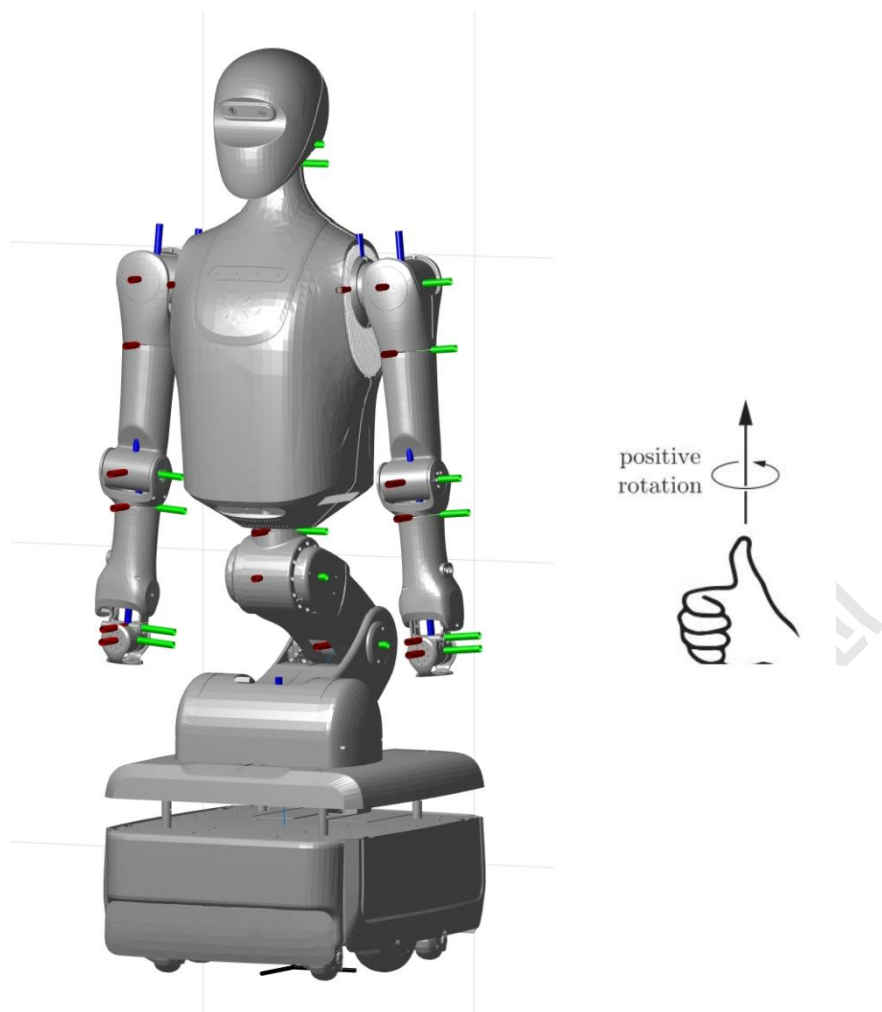


1.3.3. 旋转轴 & 正方向

1. 机器人每个旋转轴的轴线，按照上图机器人的各轴状态为基准，和机器人整机坐标系的正方向对应去寻找旋转轴的正方向;

2. 举例:

- (1) 比如腰部关节的轴线是竖直的，和整机 Z 轴 (Yaw) 靠近，所以腰部关节的正方向是向上，同构右手定则能够确认关节的选装正方向;
- (2) 比如腿部髋关节 Pitch 的轴线，和整机 Y 轴 (Pitch) 靠近，所以髋关节 Pitch 的正方向是向左，同构右手定则能够确认关节的选装正方向。



2. 快速操作指南

△ 注意：机器人目前有两种启动方式：自启动和手动程序启动。

如启用自启动，则 ✓ 需执行步骤 2.(4). 按 A 键自检，✗ 无需执行步骤 4. 程序启动；

如关闭自启动，手动程序启动，则 ✗ 无需执行步骤 2.(4). 按 A 键自检，✓ 需执行步骤 4. 程序启动。

1. 将遥控器上的电源键先短按后立即长按，打开遥控器；

2. 开机上电：

- (1) 按下总开关键；
- (2) 点按开机按钮；
- (3) 顺时针旋转急停按钮直至弹出；

- (4) **(自启动启用的前提下)** 等待机器人开机完成后（即开机音效结束），在 F 拨杆置于中间挡位的前提下，按遥控器 A 键启动自检，自检成功后（即整机状态灯变为蓝绿常亮状态）进入可操作的 Ready 状态。

3. 配置 Wi-Fi 步骤:

- (1) 用以太网线连接机器人背后的调试以太网接口和用户电脑端;
- (2) 用户电脑配置以太网端口地址为 192.168.41.xx/255.255.255.0;
- (3) 打开终端中输入 `ssh ubuntu@192.168.41.x` 登录需要配置的主控板（运控 x86 和 orin，其 IP 分别为 192.168.41.1 和 192.168.41.2）;
- (4) 在上一步的终端中继续依次输入如下命令配置 IP:

```
sudo nmcli device wifi list #查看 Wi-Fi 热点的 SSID
sudo nmcli device wifi connect 'Wi-Fi SSID' password 'Wi-Fi 密码' #连接 Wi-Fi
ifconfig #检查是否连接成功
```

- (5) Wi-Fi 配置完成，拔掉网线。

● **Wi-Fi 配置完成的前提下，每次机器人无需再配置 Wi-Fi，直接用 Wi-Fi 连接的步骤如下:**

- (1) 输入 `ifconfig`，查看给主控板配置的 IP 地址;
- (2) 输入 `ssh ubuntu@x.x.x.x` 登录连接主控板服务器，其中 `x.x.x.x` 为上一步获取的 IP 地址。随后可进行下一步程序启动。

4. **(自启动关闭的前提下) 程序启动**

- (1) 手动启动请先确保已经关闭自启动功能:

```
sudo systemctl stop proc_manager.service
```

- (2) 首先，打开第一个终端，依次输入如下命令启动**主控**节点:

```
sudo su
cd ros2ws
source install/setup.bash
ros2 launch body_control body.launch.py
```

此时可以开始使用 SDK 进行开发调试。

5. 关机:

- (1) 确认机器人已停止运动;
- (2) 按下急停按钮;
- (3) 按下主控开关键;
- (4) 按下电池开关键;
- (5) 将遥控器上的电源键先短按后立即长按, 关闭遥控器。

3. 全局值说明

3.1. 全身关节 id 定义

- 消息名称: MotorName
- 数据定义位置: MotorName.msg
- 说明: Pro 版天轶头部、臂部、腰部和腿部的电机的命名 ID 定义如下。

```
# head 00 + id
uint16 MOTOR_HEAD_1 = 1
uint16 MOTOR_HEAD_2 = 2
uint16 MOTOR_HEAD_3 = 3

# arm-left 10 + id
uint16 MOTOR_ARM_LEFT_1 = 11
uint16 MOTOR_ARM_LEFT_2 = 12
uint16 MOTOR_ARM_LEFT_3 = 13
uint16 MOTOR_ARM_LEFT_4 = 14
uint16 MOTOR_ARM_LEFT_5 = 15
uint16 MOTOR_ARM_LEFT_6 = 16
uint16 MOTOR_ARM_LEFT_7 = 17

# arm-right 20 + id
```

```
uint16 MOTOR_ARM_RIGHT_1 = 21
uint16 MOTOR_ARM_RIGHT_2 = 22
uint16 MOTOR_ARM_RIGHT_3 = 23
uint16 MOTOR_ARM_RIGHT_4 = 24
uint16 MOTOR_ARM_RIGHT_5 = 25
uint16 MOTOR_ARM_RIGHT_6 = 26
uint16 MOTOR_ARM_RIGHT_7 = 27

# body (including waist) 30 + id
uint16 MOTOR_WAIST_01 = 31
uint16 MOTOR_WAIST_02 = 32

# leg
uint16 MOTOR_LEG_LEFT_1 = 51
uint16 MOTOR_LEG_LEFT_2 = 52
```

4. 接口说明

4.1. 一键启动

4.1.1. 功能启用/关闭

自启动功能默认被禁用，使用以下方法开启、关闭

1. 使能自启动

```
sudo systemctl enable proc_manager.service
```

2. 禁用自启动

```
sudo systemctl disable proc_manager.service
```

4.2. 关节温度获取接口

- 说明：所有关节温度获取，以 topic 形式发布，每秒 1 次；

- 数据定义位置: `bodyctrl_msgs::msg::MotorStatusMsg1`
- 数据格式:

```
std_msgs/Header header
MotorStatus1[] status
```

MotorStatus1 定义如下:

```
uint16 name # MotorName
float32 motortemperature
float32 mostemperature
```

1. 获取头部关节温度信息

话题名称: `/head/motor_status`

2. 获取腰部关节温度信息

话题名称: `/waist/motor_status`

3. 获取双臂关节温度信息

话题名称: `/arm/motor_status`

4. 获取腿部关节温度信息

话题名称: `/leg/motor_status`

4.3. 头部关节

4.3.1. 状态获取接口

1. 获取头部关节信息

- 说明: 获取头部关节的状态信息, 其中包含关节的当前位置、速度、电流和温度。
- 控制方式: topic
- 话题名称: `/head/status`
- 数据格式:

```
# bodyctrl_msgs/msg/MotorStatusMsg.msg
std_msgs/Header header
MotorStatus[] status
```

```
# bodyctrl_msgs/msg/MotorStatus.msg

uint16 name          # MotorName

float32 pos           # rad

float32 speed         # rad

float32 current       # A

float32 temperature   # Mos 温度

uint32 error
```

error 错误字段值定义如下：

```
error = 33072,      设备掉线
error = 33073,      关节位置超限
error = 1,          关节电机过温
error = 2,          过流
error = 3,          电压过低
error = 4,          关节 mos 过温
error = 5,          堵转
error = 6,          电压过高
error = 7,          缺相
error = 8,          编码器错误
```

4.3.2. 控制接口

1. 位置模式

- 说明：头部关节的位置控制接口，需要提供期望位置、期望速度、最大电流。
- 控制方式：topic
- 话题名称：/head/cmd_pos
- 数据定义位置：bodyctrl_msgs::msg::CmdSetMotorPosition.msg
- 数据格式：

```
std_msgs/Header header
SetMotorPosition[] cmds
```

```
# SetMotorPosition.msg
uint16 name # MotorName
float32 pos # rad
float32 spd # rad/s
float32 cur # A
```

2. 力位混合模式

- 说明：头部关节的力位混合控制接口，需要提供期望位置、期望速度、前馈力矩、kp、kd 系数。
- 控制方式：topic
- 话题名称：/head/cmd_ctrl
- 数据定义位置：bodyctrl_msgs::msg::CmdMotorCtrl.msg
- 数据格式：

```
std_msgs/Header header
MotorCtrl[] cmds

# MotorCtrl.msg
uint16 name
float32 kp
float32 kd
float32 pos
float32 spd
float32 tor
```

3. 速度模式

- 说明：关节的速度控制接口，需要提供期望速度、最大电流。
- 控制方式：topic
- 话题名称：/head/cmd_vel
- 数据定义位置：bodyctrl_msgs::msg::CmdSetMotorSpeed.msg
- 数据格式：

```
std_msgs/Header header
```

```
SetMotorSpeed[] cmds
```

```
#SetMotorSpeed.msg
```

```
uint16 name # MotorName
```

```
float32 spd # rad/s
```

```
float32 cur # A
```

4. 关节标零

- 说明：头部关节的标零接口。
- 控制方式：topic
- 话题名称：/head/cmd_set_zero
- 数据定义位置：std_msgs::msg::String
- 数据格式：

以字符串形式发送关节的 id 实现标零；

头部关节 id：1,2,3；

4.4. 腰部关节

4.4.1. 状态获取接口

1. 获取腰部关节信息

- 说明：获取腰部关节的状态信息，其中包含关节的当前位置、速度、电流和温度。
- 控制方式：topic
- 话题名称：/waist/status
- 数据定义位置：bodyctrl_msgs::msg::MotorStatusMsg.msg
- 数据格式：

```
std_msgs/Header header
```

```
MotorStatus[] status
```

MotorStatus 定义如下：

```
uint16 name    # MotorName
float32 pos     # rad
float32 speed   # rad
float32 current # A
float32 temperature #
uint32 error
```

error 错误字段值定义如下：

```
error = 33072,    设备掉线
error = 33073,    关节位置超限
error = 1,        关节电机过温
error = 2,        过流
error = 3,        电压过低
error = 4,        关节 mos 过温
error = 5,        堵转
error = 6,        电压过高
error = 7,        缺相
error = 8,        编码器错误
```

4.4.2. 控制接口

1. 位置模式

- 说明：关节的位置控制接口，需要提供期望位置、期望速度、最大电流。
- 控制方式：topic
- 话题名称：/waist/cmd_pos
- 数据定义位置：bodyctrl_msgs::msg::CmdSetMotorPosition.msg
- 数据格式：

```
std_msgs/Header header
SetMotorPosition[] cmds

# SetMotorPosition.msg
uint16 name    # MotorName
```

```
float32 pos # rad
float32 spd # rad/s
float32 cur # A
```

2. 腰部关节标零

- 说明：腰部关节的标零接口。
- 控制方式：topic
- 话题名称：/waist/cmd_set_zero
- 数据定义位置：std_msgs::msg::String
- 数据格式：

以字符串形式发送关节的 id 实现标零；
腰部关节 id：31

4.5. 双臂关节

4.5.1. 状态获取接口

1. 获取双臂关节信息

- 说明：获取双臂关节的状态信息，其中包含关节的当前位置、速度、电流和温度。
- 控制方式：topic
- 话题名称：/arm/status
- 数据定义位置：bodyctrl_msgs::msg::MotorStatusMsg.msg
- 数据格式：

```
std_msgs/Header header
MotorStatus[] status
```

MotorStatus 定义如下：

```
uint16 name # MotorName
float32 pos # rad
float32 speed # rad
float32 current # A
```

```
float32 temperature
uint32 error
```

error 错误字段值定义如下:

```
error = 33072,    设备掉线
error = 33073,    关节位置超限
error = 1,        关节电机过温
error = 2,        过流
error = 3,        电压过低
error = 4,        关节 mos 过温
error = 5,        堵转
error = 6,        电压过高
error = 7,        缺相
error = 8,        编码器错误
```

4.5.2. 控制接口

1. 位置模式

- 说明: 关节的位置控制接口, 需要提供期望位置、期望速度、最大电流。
- 控制方式: topic
- 话题名称: `/arm/cmd_pos`
- 数据定义位置: `bodyctrl_msgs::msg::CmdSetMotorPosition.msg`
- 数据格式:

```
std_msgs/Header header
SetMotorPosition[] cmds

# SetMotorPosition.msg
uint16 name # MotorName
float32 pos # rad
float32 spd # rad/s
float32 cur # A
```

2. 力位混合模式

- 说明：关节的力位混合控制接口，需要提供期望位置、期望速度、前馈力矩、kp、kd 系数。
- 控制方式：topic
- 话题名称：/arm/cmd_ctrl
- 数据定义位置：bodyctrl_msgs::msg::CmdMotorCtrl.msg
- 数据格式：

```
std_msgs/Header header
MotorCtrl[] cmds
```

```
# MotorCtrl.msg
uint16 name
float32 kp
float32 kd
float32 pos
float32 spd
float32 tor
```

3. 速度模式

- 说明：关节的速度控制接口，需要提供期望速度、最大电流。
- 控制方式：topic
- 话题名称：/arm/cmd_vel
- 数据定义位置：bodyctrl_msgs::msg::CmdSetMotorSpeed.msg
- 数据格式：

```
std_msgs/Header header
SetMotorSpeed[] cmds
```

```
#SetMotorSpeed.msg
uint16 name # MotorName
float32 spd # rad/s
float32 cur # A
```

4. 电流模式

- 说明：关节的电流控制接口，需要提供期望电流。
- 控制方式：topic
- 话题名称：/arm/cmd_current
- 数据定义位置：bodyctrl_msgs::msg::CmdSetMotorCurTor.msg
- 数据格式：

```
std_msgs/Header header
SetMotorCurTor[] cmds

#SetMotorCurTor.msg
uint16 name # MotorName
float32 cur_tor
uint8 ctrl_status    #当前只支持电流模式
```

5. 手臂关节标零

- 说明：双臂关节的标零接口。
- 控制方式：topic
- 话题名称：/arm/cmd_set_zero
- 数据定义位置：std_msgs::msg::String
- 数据格式：

以字符串形式发送关节的 id 实现标零；

双臂关节 id：

左臂：11---17,

右臂：21---27；

4.6. 腿部关节

4.6.1. 状态获取接口

1. 获取双腿关节信息

- 说明：获取双腿关节的状态信息，其中包含关节的当前位置、速度、电流、温度和告警。

- 控制方式: topic
- 话题名称: /leg/status
- 数据定义位置: bodyctrl_msgs::msg::MotorStatusMsg.msg
- 数据格式:

```
std_msgs/Header header
MotorStatus[] status
```

MotorStatus 定义如下:

```
uint16 name      # MotorName
float32 pos      # rad
float32 speed    # rad
float32 current  # A
float32 temperature
uint32 error
```

error 错误字段值定义如下:

```
error = 33072,    设备掉线
error = 33073,    关节位置超限
error = 1,        关节电机过温
error = 2,        过流
error = 3,        电压过低
error = 4,        关节 mos 过温
error = 5,        堵转
error = 6,        电压过高
error = 7,        缺相
error = 8,        编码器错误
```

4.6.2. 控制接口

1. 位置模式

- 说明: 关节的位置控制接口, 需要提供期望位置、期望速度、最大电流。
- 控制方式: topic

- 话题名称: `/leg/cmd_pos`
- 数据定义位置: `bodyctrl_msgs::msg::CmdSetMotorPosition.msg`
- 数据格式:

```
std_msgs/Header header
SetMotorPosition[] cmds

# SetMotorPosition.msg
uint16 name # MotorName
float32 pos # rad
float32 spd # rad/s
float32 cur # A
```

4.7. 手部关节

4.7.1. 全局值说明

- 消息名称: `MotorName`
- 数据定义位置:
- 说明: 手指各关节 ID 定义如下:

```
uint16 MOTOR_FINGER_LITTLE = 1, //little finger 小指
uint16 MOTOR_FINGER_RING = 2, //ring finger 无名指
uint16 MOTOR_FINGER_MIDDLE = 3, //middle finger 中指
uint16 MOTOR_FINGER_FORE = 4, //Fore finger 食指
uint16 MOTOR_FINGER_THUMB_BEND = 5, //thumb bend 拇指弯曲
uint16 MOTOR_FINGER_THUMB_ROTATION = 6, //thumb rotation 拇指旋转
```

4.7.2. 状态获取接口

- 说明: 获取手指各关节的状态信息, 其中包含电机的当前位置、速度、电流。
- 控制方式: topic
- 话题名称: `/inspire_hand/state/left_hand` 和 `/inspire_hand/state/right_hand`
- 数据定义位置: `sensor_msgs::msg::JointState`

-  **注意：** 这里的数据(位置，速度，电流)，指的是百分比

```
std_msgs/Header header

string[] name          #说明，id=1 为小指，id=6 为大拇指旋转

float64[] position     #百分比

float64[] velocity     #百分比

float64[] effort       #百分比
```

4.7.3. 控制接口-topic

1. 控制手指各关节角度

- 说明：手部电机的位置控制接口，需要提供期望位置、期望速度、最大电流。
- 控制方式：topic
- 话题名称：/inspire_hand/ctrl/left_hand 和 /inspire_hand/ctrl/right_hand
- 数据定义位置：sensor_msgs::msg::JointState.msg
-  **注意：** 这里的 position，指的仍然是手指张开的百分比

```
std_msgs/Header header

string[] name

float64[] position     #百分比
```

4.7.4. 控制接口-service

1. 控制手指各关节角度

- 说明：手指各个关节的位置控制接口，以百分比的方式。
- 控制方式：service
- 服务名称：/inspire_hand/set_angle_flexible/left_hand 和 /inspire_hand/set_angle_flexible/right_hand
- 数据定义位置：bodyctrl_msgs::srv::set_angle_flexible.srv
- 数据格式：

```
string[] name

float32[] angle_ratio
```

bool angle_accepted

4.7.5. 控制手指各关节力矩

- 说明：手指各个关节的位置力矩，以百分比的方式。
- 控制方式：service
- 服务名称：/inspire_hand/set_force/left_hand 和 /inspire_hand/set_force/right_hand
- 数据定义位置：bodyctrl_msgs::srv::set_force.srv
- request:

```
float32 force0_Ratio  #小拇指-百分比 (0 代表 0g, 1 代表 1000g)
float32 force1_Ratio  #无名指-百分比
float32 force2_Ratio  #中指-百分比
float32 force3_Ratio  #食指-百分比
float32 force4_Ratio  #大拇指弯曲-百分比
float32 force5_Ratio  #大拇指旋转-百分比
```

- response:

bool force_accepted #是否设置成功

4.7.6. 控制手指各关节速度

- 说明：手指各个关节的位置力矩，以百分比的方式。
- 控制方式：service
- 服务名称：/inspire_hand/set_speed/left_hand 和 /inspire_hand/set_speed/right_hand
- 数据定义位置：bodyctrl_msgs::srv::set_speed.srv
- request:

```
float32 speed0_Ratio  #小拇指-百分比 (1 代表 800ms 从最大角度到最小角度, 0.5
代表 1600ms, 0.25 代表 3200ms)
float32 speed1_Ratio  #无名指-百分比
```

```
float32 speed2_Ratio  #中指-百分比  
float32 speed3_Ratio  #食指-百分比  
float32 speed4_Ratio  #大拇指弯曲-百分比  
float32 speed5_Ratio  #大拇指旋转-百分比
```

- response:

```
bool speed_accepted  #是否设置成功
```

4.7.7. 清除手指各关节错误

- 说明：清除手指各个关节的错误接口。
- 控制方式：service
- 服务名称：/inspire_hand/set_clear_error/left_hand 和 /inspire_hand/set_clear_error/right_hand
- 数据定义位置：bodyctrl_msgs::srv::set_clear_error.srv
- request:

```
null
```

- response:

```
bool setclear_error_accepted  #是否设置成功
```

4.8. 六维力

4.8.1. 状态获取接口

1. 六维力信息

- 说明：获取六维力当前的信息。其中上报频率为 1000Hz。
- 控制方式：topic
- 话题名称：/arm_6dof_left 和 /arm_6dof_right
- 类型：geometry_msgs::msg::WrenchStamped.msg
- 数据格式：

```
Header header uint32 seq #序列号
```

Header header time stamp #时间戳

Header header string frame_id #帧 id

Wrench wrench Vector3 force #3 个方向的力

Wrench wrench Vector3 torque #3 个方向的力矩

4.9. lyre 语音包

4.9.1. 程序包及安装位置

lyre 语音包包括 lyre_ros2、lyre_msgs 和 lyre_examples 三个程序包。

- lyre_ros2 是语音包的功能程序包，安装在 orin 主控中。
- lyre_msgs 是语音包的接口定义。该接口定义同时安装在 x86 的 ros2ws 目录以及 lyre_ros2 目录。
- lyre_examples 是语音包接口的调用示例程序。该示例程序安装在 x86 的 ros2ws 目录。

lyre 语音包程序的目录结构如下：

```
├── ros2ws/                # x86
│   ├── install
│   │   ├── lyre_msgs      # 接口定义
│   │   │   ├── README.md  # SDK 开发文档
│   │   │   └── lyre_examples # 接口调用示例程序
│   └──
├── lyre_ros2/            # orin
│   ├── install
│   │   ├── lyre_msgs      # 接口定义
│   │   │   ├── README.md  # SDK 开发文档
│   │   │   └── lyre        # 功能包
│   ├── scripts
│   │   ├── setup_network.sh # 网络配置脚本
│   │   ├── setup_systemd.sh # 自启动服务部署脚本
│   │   └── reset_systemd.sh # 自启动服务卸载脚本
```

| |— README.md # 用户手册
| |— QAs.md # 常见问题

4.9.2. 语音交互功能

机器人提供有语音交互功能。设备开机以后，语音交互功能为关闭状态。通过遥控器按键可以开启语音交互功能。

4.9.2.1. 遥控器开关

E、G、H 键在中间； F 键向上。

长按 A 键，可以打开/关闭语音交互功能。

4.9.2.2. 语音对话方式

站立在机器人正前方 1.5 米左右的位置，正视机器人胸前麦克风上方的摄像头，与机器人进行对话。

4.9.3. 启动语音包

语音包的 ROS 包名为 lyre，在 systemd 中的服务名称也是 lyre。

4.9.3.1. 手动启动语音对话的 ROS 节点

在 lyre_ros2 程序包中使用下面的命令启动语音对话的 ROS 节点。

```
cd lyre_ros2
source install/setup.bash
ros2 launch lyre chat.launch.py
```

注意：机器人默认会在开始时自行启动 lyre 节点。同一时间只允许一个 lyre 进程运行。通过下面的命令检查系统中是否有其它 lyre 进程正在运行。

```
ps -ef | grep "ros2 launch lyre" | grep -v grep
```

如果有打印内容，则说明 lyre 进程正在运行。通过下面的命令可以关闭系统中正在运行的 lyre 进程。

```
pkill -INT -f "ros2 launch lyre"
```

注意：如果通过 systemd 配置了开机自启动服务，则需要在关闭自启动服务以后再结束进

程。否则，手动结束的进程会被重新启动。具体操作参考后面的章节。

4.9.3.2. 开机自启动服务

出厂时，语音包的 `systemd` 服务 `lyre` 会在开机时自行启动。

检查 `lyre` 服务是否会在开机时自启动

```
systemctl is-enabled lyre
```

如果显示 `enabled`，则说明 `lyre` 服务会在开机时自启动。

打开 `lyre` 服务开机自启动设置

```
sudo systemctl enable lyre
```

关闭 `lyre` 服务开机自启动设置

```
sudo systemctl disable lyre
```

4.9.4. 启动模式

`lyre` 语音包提供了四种运行模式，分别是 `play`、`asr`、`audio`、`chat`。接入者可以根据实际情况选用其中的一种。

注意：启动模式是互斥的。同一时间，只可以允许 4 种启动模式中的一种。

- `play`: 在此启动模式下，仅『音频播放』相关的功能可以使用。
- `asr`: 在此启动模式下，仅『语音识别』相关的功能可以使用。
- `audio`: 在此启动模式下，可以同时使用『语音识别』和『音频播放』功能，包括『文本播放』功能。
- `chat`: 在此启动模式下，可以同时使用『语音识别』、『音频播放』和『语音对话』功能。

不同模式的节点的启动命令统一为：

```
ros2 launch lyre <mode>.launch.py
```

例如，启动 `audio` 模式的命令为：

```
ros2 launch lyre audio.launch.py
```

部署某个启动模式的开机自启动服务

lyre_ros2 程序包给出了自启动服务的部署脚本 `setup_systemd.sh`，该脚本位于 `lyre_ros2/scripts` 目录下。使用方法如下：

```
bash setup_systemd.sh <mode>[default:chat]
```

例如：将 `play` 启动模式部署为自启动服务的命令如下：

```
bash setup_systemd.sh play
```

4.9.5. 接口示例程序

lyre_examples 程序包提供了部分 C++ 版本的接口调用示例。程序包的目录结构如下：

```
lyre_examples/
├── bin/                                # 接口调用示例的可执行程序
│   ├── lyre_example_play_file_srv
├── cli/                                # 接口调用示例的终端执行脚本
│   ├── call_play_file_srv.sh
│   ├── sub_play_progress_msg.sh
├── src/                                # 接口调用示例的 C++ 代码
│   ├── play_file_srv.cpp
├── res/                                # 静态资料
│   ├── test.mp3
├── CMakeLists.txt                      # 项目 CMakeLists.txt 示例
└── package.xml                         # 项目 ROS package.xml 示例
```

对于每个开放的语音接口，lyre_examples 包都提供有接口调用的示例代码、编译后的可执行程序、终端调用脚本，分别放置在 `src`、`bin`、`cli` 目录。

为了避免混淆，每个演示所用的可执行程序的文件名前面都加了 `lyre_example_` 作为前缀。每个用于订阅 Topic 的脚本的文件名都加了 `sub_` 作为前缀；每个用于发布 Topic 的脚本的文件名都加了 `pub_` 作为前缀。每个用于调用 Service 的脚本的文件名则都加了 `call_` 作为前缀。

对于二次开发的 C++ ROS2 项目，可以参考 lyre_examples 包中的方式导入 lyre_msgs 接口，再参考示例代码完成功能开发。

4.9.5.1. ROS package 配置

```
<package format="3">
  <depend>lyre_msgs</depend>
</package>
```

4.9.5.2. 项目 CMake 配置

```
find_package(lyre_msgs REQUIRED)

ament_target_dependencies(
  <your_target>
  lyre_msgs
)
```

4.9.5.3. 编译时、运行时依赖配置

x86 的 `ros2ws` 和 `orin` 的 `lyre_ros2` 提供了 `lyre_msgs` 包。在构建或者运行自己的 ROS 程序之前，需要先从这两个工作空间导入 `lyre_msgs` 依赖。

```
# x86
source ros2ws/install/setup.bash

# orin
source lyre_ros2/install/setup.bash
```

4.9.6. 音频播放接口

4.9.6.1. 播放本地文件 (Service)

如果你已经明确知道 `lyre_ros2` 程序包所在的主控板中有一个音频文件，你可以使用下面的 ROS 服务播放这个文件。

```
# lyre_msgs/srv/PlayFile.srv
# 以本地文件的形式播放一段语音。

string SERVICE_NAME = /audio_play/play_file
```

```

string sid    # Stream identifier (unique per audio stream).
uint32 seq    # Sequence number (incremental per packet).
bool  last    # Last flag (true if this is the final packet).
bool  force   # Force playback (stop all running tasks and play immediately).

string path   # Absolute path to the audio file in the local file system.
---

int8 CODE_OK = 0
int8 CODE_INVALID_PARAMS = 1
int8 CODE_FAILED = -1

string sid     # Playback stream ID (generated internally if absent).
int8  code     # Status code.
string message  # Human-readable status message.

```

接口调用的 C++ 示例代码请参考 lyre_examples 程序包中的 play_text_srv。

4.9.6.2. 播放远程文件 (Service)

你还可以使用下面的 ROS 服务来播放一个来自网络的音频文件。

```

# lyre_msgs/srv/PlayUrl.srv
# 以文件 URL 的形式播放一段语音。

string SERVICE_NAME = /audio_play/play_url

string sid    # Stream identifier (unique per audio stream).
uint32 seq    # Sequence number (incremental per packet).
bool  last    # Last flag (true if this is the final packet).
bool  force   # Force playback (stop all running tasks and play immediately).

string url    # URL to the audio file in the network.
---

```

```

int8 CODE_OK = 0
int8 CODE_INVALID_PARAMS = 1
int8 CODE_FAILED = -1

string sid      # Playback stream ID (generated internally if absent).
int8  code      # Status code.
string message   # Human-readable status message.

```

接口调用的 C++ 示例代码请参考 lyre_examples 程序包中的 play_url_srv。

4.9.6.3. 播放文本 (Service)

如果你需要从别的主控板访问音频播放服务,可以使用下面的 ROS 服务来播放一个字符串。

```

# lyre_msgs/srv/PlayText.srv
# 以文本的形式播放一段语音。

string SERVICE_NAME = /audio_play/play_text

string sid      # Stream identifier (unique per audio stream).
uint32 seq      # Sequence number (incremental per packet).
bool  last      # Last flag (true if this is the final packet).
bool  force     # Force playback (stop all running tasks and play immediately).

string text     # Text to be synthesized into speech.

string token     # System api (unavailable for applications).
string output    # System api (unavailable for applications).
---

int8 CODE_OK = 0
int8 CODE_INVALID_PARAMS = 1
int8 CODE_FAILED = -1

```

```
string sid      # Playback stream ID (generated internally if absent).
int8  code      # Status code.
string message  # Human-readable status message.
```

接口调用的 C++ 示例代码请参考 lyre_examples 程序包中的 play_text_srv。

注意：此功能仅在 audio 和 chat 启动模式下才可用。

4.9.6.4. 监听播放事件 (Topic)

播放音频的过程中会通过下面的 Topic 来发布播放事件：

```
# lyre_msgs/msg/PlayEvent.msg
# Play 音频播放的事件。

string TOPIC_NAME = /audio_play/event

int8 EVENT_STARTED    = 0
int8 EVENT_COMPLETED = 1
int8 EVENT_STOPPED    = 2
int8 EVENT_CANCELLED  = 3
int8 EVENT_FAILED     = 4

string sid
uint32 seq
int8  event
string message
```

接口调用的 C++ 示例代码请参考 lyre_examples 程序包中的 play_event_msg。

4.9.6.5. 监听播放进度 (Topic)

播放音频的过程中会通过下面的 Topic 来发布播放的进度及总时长（单位：秒）：

```
# lyre_msgs/msg/PlayProgress.msg
# Play 音频播放的进度。
```

```
string TOPIC_NAME = /audio_play/progress
```

```
string sid
```

```
uint32 seq
```

```
float64 position
```

```
float64 duration
```

接口调用的 C++ 示例代码请参考 lyre_examples 程序包中的 play_progress_msg。

4.9.6.6. 停止音频播放 (Service)

通过下面的 ROS 服务来停止音频播放（停止以后无法恢复）：

```
# lyre_msgs/srv/PlayStop.srv
```

```
# 停止当前的音频播放任务（不可恢复）。
```

```
string SERVICE_NAME = /audio_play/stop
```

```
---
```

接口调用的 C++ 示例代码请参考 lyre_examples 程序包中的 play_stop_srv。

4.9.6.7. 暂停音频播放 (Service)

通过下面的 ROS 服务来停止音频播放（暂停以后可以调用 Resume 服务恢复播放）：

```
# lyre_msgs/srv/PlayPause.srv
```

```
# 暂停当前的音频播放任务。
```

```
string SERVICE_NAME = /audio_play/pause
```

```
---
```

接口调用的 C++ 示例代码请参考 lyre_examples 程序包中的 play_pause_srv。

4.9.6.8. 恢复音频播放 (Service)

通过下面的 ROS 服务来恢复播放暂停中的音频：

```
# lyre_msgs/srv/PlayResume.srv
# 恢复已暂停的音频播放任务。

string SERVICE_NAME = /audio_play/resume

---
```

接口调用的 C++ 示例代码请参考 lyre_examples 程序包中的 play_resume_srv。

4.9.7. 语音识别接口

设备集成有语音识别模块，可以提供 ASR 语音识别功能。

4.9.7.1. 监听唤醒事件 (Topic)

在设备周围喊出关键词，可以触发唤醒事件。不同设备的唤醒关键词请参考用户手册。

在代码中可以通过以下 ROS 话题来监听唤醒消息：

```
# lyre_msgs/msg/AsrKeyword.msg
# ASR 唤醒时识别到的关键字。

string TOPIC_NAME = /audio_asr/keyword

string keyword
int32 angle
```

接口调用的 C++ 示例代码请参考 lyre_examples 程序包中的 asr_keyword_msg。

4.9.7.2. 监听语音识别结果 (Topic)

设备可以将周围的声音转化为文本。可以通过以下 ROS 话题获取识别到的文本：

```
# lyre_msgs/msg/Asrlat.msg
# ASR 语音识别到的内容。

string TOPIC_NAME = /audio_asr/iat
```

```
string id
string text
```

接口调用的 CPP 示例代码请参考 lyre_examples 程序包中的 asr_iat_msg。

4.9.7.3. 监听其它事件 (Topic)

设备还会发布其它 ASR 相关的事件，事件类型定义在下面的 ROS 话题中：

```
# lyre_msgs/msg/AsrEvent.msg
# ASR 设备报出的事件。

string TOPIC_NAME = /audio_asr/event

int8 EVENT_ERROR    = 2  # 出错事件。arg1 是错误码。
int8 EVENT_STATE    = 3  # 服务状态事件。
int8 EVENT_WAKEUP   = 4  # 唤醒事件。
int8 EVENT_SLEEP    = 5  # 休眠事件。

int8 EVENT_PRE_SLEEP      = 10 # 准备休眠事件。
int8 EVENT_CONNECTED_TO_SERVER = 13 # 与服务端建立连接。
int8 EVENT_SERVER_DISCONNECTED = 14 # 与服务端断开连接。

int8 event
int8 arg1
```

接口调用的 CPP 示例代码请参考 lyre_examples 程序包中的 asr_event_msg。

4.9.7.4. aiui_msg 事件

SDK 兼容旧版本的 /xunfei/aiui_msg 话题，这个话题已经被标记为 Deprecated，未来很可能会被移除。

```
string TOPIC_NAME = /xunfei/aiui_msg
```

该话题的类型是 std_msgs::msg::String，可以通过下面的示例脚本获取到：

```
ros2 topic echo -f /xunfei/aiui_msg
```

4.9.8. 语音对话接口

4.9.8.1. 语音交互开关

lyre 节点接收一个 ROS 话题来控制语音交互的可用状态。

```
string TOPIC_NAME = /audio_chat/enable
```

话题的类型是 std_msgs/msg/Bool。

对话开启的示例脚本：

```
ros2 topic pub -1 /audio_chat/enable std_msgs/msg/Bool 'data: true'
```

对话关闭的示例脚本：

```
ros2 topic pub -1 /audio_chat/enable std_msgs/msg/Bool 'data: false'
```

4.10. 深度相机

启动奥比相机方法-ROS2

- Orin (192.168.41.2) :

```
cd /home/nvidia/orbbec_camera_ros2
source install/setup.bash
ros2 launch orbbec_camera gemini_330_series.launch.py
```

4.11. 电池状态及电量获取

1. 电池状态

- 说明：获取电池状态信息。其中上报频率为 1Hz。
- 充电状态：master_battery_current 字段，负值表示在放电，正值表示在充电
- 控制方式：topic
- 话题名称：/power/battery/status
- 类型：bodyctrl_msgs::msg::PowerBatteryStatus
- 成员：

@brief 电源板电压、电流、电量、温度等信息

std_msgs/Header header

battery status information

int32 battery_installed # 0x00 都不存在, 0x01 小电池存在, 0x02 大电池存在, 0x03
大小电池都存在

int32 battery_working # 0x10 大电池, 0x01 小电池

float32 master_battery_voltage # 电池电压信息

float32 master_battery_current # 电池电流信息, 负值表示在放电, 正值表示在充电

float32 master_battery_power # 电池电量

float32 little_battery_voltage # 小电池电压信息

float32 little_battery_current # 小电池电流信息, 负值表示在放电, 正值表示在充电

float32 little_battery_power # 小电池电量

int8 pg12a # 00 低, 01 高

int8 pg12b # 00 低, 01 高

int8 pg12c # 00 低, 01 高

int8 pg12d # 00 低, 01 高

int8 pg5cd # 00 低, 01 高

int8 pg5ab # 00 低, 01 高

int8 pgrdc2 # 00 低, 01 高

int8 pgrdc1 # 00 低, 01 高

int8 pgheader # 00 低, 01 高

int8 pgbutton2 # 00 低, 01 高

2. 电源板状态

- 说明：获取电源板状态信息。其中上报频率为 1Hz。
- 控制方式：topic
- 话题名称：/power/board/status
- 类型：bodyctrl_msgs::msg::PowerStatus
- 成员：

@brief 电源板电压、电流、电量、温度等信息

std_msgs/Header header

temperature information

float32 waist_temp # 腰部 MOS 温度

float32 arm_a_temp # 臂 A MOS 温度

float32 arm_b_temp # 臂 B MOS 温度

float32 leg_a_temp # 腿 A MOS 温度

float32 leg_b_temp # 腿 B MOS 温度

float32 waist_temp_max # 腰部 MOS 温度最大值

float32 arm_a_temp_max # 臂 A MOS 温度最大值

float32 arm_b_temp_max # 臂 B MOS 温度最大值

float32 leg_a_temp_max # 腿 A MOS 温度最大值

float32 leg_b_temp_max # 腿 B MOS 温度最大值

float32 waist_temp_min # 腰部 MOS 温度最小值

float32 arm_a_temp_min # 臂 A MOS 温度最小值

float32 arm_b_temp_min # 臂 B MOS 温度最小值

float32 leg_a_temp_min # 腿 A MOS 温度最小值

float32 leg_b_temp_min # 腿 B MOS 温度最小值

current information

float32 arm_a_curr # 臂 A 电流

float32 arm_b_curr # 臂 B 电流

float32 leg_a_curr # 腿 A 电流

float32 leg_b_curr # 腿 B 电流

float32 waist_curr # 腰部电流

float32 head_curr # 头部电流

float32 arm_a_curr_max # 臂 A 电流最大值

float32 arm_b_curr_max # 臂 B 电流最大值

float32 leg_a_curr_max # 腿 A 电流最大值

float32 leg_b_curr_max # 腿 B 电流最大值

float32 waist_curr_max # 腰部电流最大值

float32 head_curr_max # 头部电流最大值

float32 arm_a_curr_min # 臂 A 电流最小值

float32 arm_b_curr_min # 臂 B 电流最小值

float32 leg_a_curr_min # 腿 A 电流最小值

float32 leg_b_curr_min # 腿 B 电流最小值

float32 waist_curr_min # 腰部电流最小值

float32 head_curr_min # 头部电流最小值

float32 arm_a_volt # 臂 A 电压

float32 arm_b_volt # 臂 B 电压

```
float32 leg_a_volt #腿 A 电压
float32 leg_b_volt #腿 B 电压
float32 waist_volt #腰部电压
float32 bus_volt    #母线电压

float32 arm_a_volt_max #臂 A 电压最大值
float32 arm_b_volt_max #臂 B 电压最大值
float32 leg_a_volt_max #腿 A 电压最大值
float32 leg_b_volt_max #腿 B 电压最大值
float32 waist_volt_max #腰部电压最大值
float32 bus_volt_max #母线电压最大值

float32 arm_a_volt_min #臂 A 电压最小值
float32 arm_b_volt_min #臂 B 电压最小值
float32 leg_a_volt_min #腿 A 电压最小值
float32 leg_b_volt_min #腿 B 电压最小值
float32 waist_volt_min #腰部电压最小值
float32 bus_volt_min #母线电压最小值

# version information
string software_version # 软件版本
string hardware_version # 硬件版本

# battery status information
float32 battery_voltage # 电池电压信息
float32 battery_current # 电池电流信息
float32 battery_power    # 电池电量
```

3. 急停按键状态

- 说明：获取急停按键状态信息。其中上报频率为 1Hz。
- 控制方式：topic
- 话题名称：/power/board/key_status
- 类型：bodyctrl_msgs::msg::PowerBoardKeyStatus.msg
- 成员：

```
# @brief 电源板按键状态
```

```
std_msgs/Header header
```

```
# power board status
```

```
uint32 work_time          # 工作时间
```

```
std_msgs/Bool is_estop    # 急停是否被按下
```

```
std_msgs/Bool is_remote_estop # 软急停是否被按下
```

```
std_msgs/Bool is_power_on  # 电源是否正常供电
```

4.12. 遥控器接口

1. 遥控器事件数据

- 说明：获取遥控器原始数据。
- 控制方式：topic
- 话题名称：/sbus_data/event
- 类型：bodyctrl_msgs::msg::SbusData.msg
- 数据格式：

```
int32 KEY_NONE=0
```

```
int32 KEY_A_UP=1      # A 键抬起事件
```

```
int32 KEY_A_DOWN=2    # A 键按下事件
```

```
int32 KEY_B_UP=3      # B 键抬起事件
```

```
int32 KEY_B_DOWN=4    # B 键按下事件
```

```
int32 KEY_C_UP=5      # C 键抬起事件
```

```

int32 KEY_C_DOWN=6      # C 键按下事件
int32 KEY_D_UP=7        # D 键抬起事件
int32 KEY_D_DOWN=8      # D 键按下事件
int32 KEY_E_UP=9        # E 键上拨事件
int32 KEY_E_MID=10      # E 键回中事件
int32 KEY_E_DOWN=11     # E 键下拨事件
int32 KEY_F_UP=12       # F 键上拨事件
int32 KEY_F_MID=13      # F 键回中事件
int32 KEY_F_DOWN=14     # F 键下拨事件
int32 KEY_G_LEFT=15     # G 键左拨事件
int32 KEY_G_MID=16      # G 键回中事件
int32 KEY_G_RIGHT=17    # G 键右拨事件
int32 KEY_H_LEFT=18     # H 键左拨事件
int32 KEY_H_MID=19      # H 键回中事件
int32 KEY_H_RIGHT=20    # H 键右拨事件

```

std_msgs/Header header

```

int32 key_event_new      # 按键事件新值
int32 key_event_old      # 按键事件旧值
                        # 例如 G 键从右边拨到中间位置，则
key_event_old=KEY_G_RIGHT, key_event_new=KEY_G_MID

```

```

int8 button_a           # A 键键值, -1: 松开, 1: 按下
int8 button_b           # B 键键值, -1: 松开, 1: 按下
int8 button_c           # C 键键值, -1: 松开, 1: 按下
int8 button_d           # D 键键值, -1: 松开, 1: 按下
int8 button_e           # E 键键值, <-0.5: 上拨, -0.5~0.5: 中间, >0.5: 下拨

```

int8 button_f	# F 键键值, <-0.5: 上拨, -0.5~0.5: 中间, >0.5: 下拨
int8 button_g	# G 键键值, <-0.5: 左拨, -0.5~0.5: 中间, >0.5: 右拨
int8 button_h	# H 键键值, <-0.5: 右拨, -0.5~0.5: 中间, >0.5: 左拨
float32 x1	# 左摇杆 X 方向值 (左右方向), 范围-1.0~1.0
float32 y1	# 左摇杆 Y 方向值 (上下方向), 范围-1.0~1.0
float32 x2	# 右摇杆 X 方向值 (左右方向), 范围-1.0~1.0
float32 y2	# 右摇杆 Y 方向值 (上下方向), 范围-1.0~1.0

4.13. 软件版本信息获取

完整版本信息文件 (包含 x86、orin、firmware 等完整版本信息描述文件) 定义在本体软件包的安装目录中:

ros2 路径是: /home/ubuntu/ros2ws/version_info.json

格式如下:

```
{
  "product": "TY2.0-ProD",
  "version": "0e72f5ff",
  "commit":
    "bu/integration/tianyi2.0ProD,0e72f5ffe2db4264123142ebf3bd4dff96bfdd17",
  "release_time": "2025-09-16",
  "x86": [
    {
      "module": "benti",
      "status": "enable",
      "product": "TY2.0-pro",
      "platform": "linux-x86",
      "version": "v2.0.5",
      "file_name":
        "TY2.0-pro_linux-x86_general_v2.0.5_20250912_153824.tar.gz",
      "commit":
```

```

"bu/project/tianyi2.0/prod,db8ab4dd8c2dc508a30073ce2d6e545082344abb",
  "install_dir_name": "ros2ws",
  "install_address": "192.168.41.1",
  "release_time": "2025-09-12 15:43:09"
},
{
  "module": "ota",
  "status": "enable",
  "product": "ota",
  "platform": "linux-x86",
  "version": "v2.0.7",
  "file_name": "ota_linux-x86_general_v2.0.7_20250805_211023.tar.gz",
  "commit": "bu/base/ota,d20fdc662604f6af800dc9d7b0fac40353a98997",
  "install_dir_name": "ota",
  "install_address": "192.168.41.1",
  "release_time": "2025-08-05 21:11:23"
},
{
  "module": "xlibs",
  "status": "enable",
  "product": "xlibs",
  "platform": "x86_64",
  "version": "v0.0.1",
  "file_name": "xlibs-x86_64-v0.0.1-20250815_085310.tar.gz",
  "commit":
"bu/project/common/xlibs,ca32779d7e25845286de77c015bb6216c0ce6378",
  "install_dir_name": "xlibs",
  "install_address": "192.168.41.1",
  "release_time": "2025-08-15 08:53:10"
}
]

```

```
}
```

4.14. bag 录制接口

使用 launch 文件可以启动 ros2 topic 录制, 并存储到默认路径 (/home/ubuntu/bags)。
录制按照 100M bag 分片, 录制目录最大存储容量为 2G, 超过限制后会自动删除最早文件夹以及 bag 文件。

```
# 录制指定 topic 包
# 录制列表存储在 /home/ubuntu/ros2ws/install/utils/lib/utils/topics.config
# 默认列表中存储了本体控制的关键 topic
ros2 launch utils record_config_topic.py

# 录制全量 topic 包
ros2 launch utils record_all_topic.py
```

4.15. 蓝牙模块

机器人支持通过蓝牙实现设备控制与状态监控等, 具体内容详见[诊断及蓝牙接入接口.pdf](#)。

4.16. 导航接口

<https://wiki.slamtec.com/display/SD/Slamware+Solution>

<https://wiki.slamtec.com/display/SD/KBSW180113+RoboStudio+User+Manual>

4.17. 底盘接口

底盘目前提供两种控制方式:

1. rostopic 控制方式 :

https://developer.slamtec.com/docs/slamware/ros-sdk/2.8.2_rtm/slamware_ros_sdk_server_node/

<https://slamtec.feishu.cn/docx/IV5edQISqoqGuXx8wD3cTNA2nMg>

2. http 控制方式 :

<https://docs.slamtec.com>

主机设置 192.168.11/24 网段, 浏览器访问 192.168.11.1:1448